

```
    struct Point* p = malloc(sizeof(struct Point));
    p->x = x;
    p->y = y;
    return p;
}

double distance(struct Point* p1, struct Point* p2) {
    double dx = p1->x - p2->x;
    double dy = p1->y - p2->y;
    return sqrt(dx*dx+dy*dy);
}
```

The users of `point.h` have no access whatsoever to the members of `struct Point`. They can call the `makePoint()` function, and the `distance()` function, but they have absolutely no knowledge of the implementation of either the `Point` data structure or the functions.

This is perfect encapsulation—in a non-OO language. C programmers used to do this kind of thing all the time. We would forward declare data structures and functions in header files, and then implement them in implementation files. Our users never had access to the elements in those implementation files.

But then came OO in the form of C++—and the perfect encapsulation of C was broken.

The C++ compiler, for technical reasons,¹ needed the member variables of a class to be declared in the header file of that class. So our `Point` program changed to look like this:

[Click here to view code image](#)

`point.h`

```
class Point {
public:
    Point(double x, double y);
    double distance(const Point& p) const;

private:
    double x;
    double y;
};
```

[Click here to view code image](#)

point.cc

```
#include "point.h"
#include <math.h>

Point::Point(double x, double y)
: x(x), y(y)
{}

double Point::distance(const Point& p) const {
    double dx = x-p.x;
    double dy = y-p.y;
    return sqrt(dx*dx + dy*dy);
}
```

Clients of the header file `point.h` know about the member variables `x` and `y`! The compiler will prevent access to them, but the client still knows they exist. For example, if those member names are changed, the `point.cc` file must be recompiled! Encapsulation has been broken.

Indeed, the way encapsulation is partially repaired is by introducing the `public`, `private`, and `protected` keywords into the language. This, however, was a *hack* necessitated by the technical need for the compiler to see those variables in the header file.

Java and C# simply abolished the header/implementation split altogether, thereby weakening encapsulation even more. In these languages, it is impossible to separate the declaration and definition of a class.

For these reasons, it is difficult to accept that OO depends on strong encapsulation. Indeed, many OO languages² have little or no enforced encapsulation.

OO certainly does depend on the idea that programmers are well-behaved enough to not circumvent encapsulated data. Even so, the languages that claim to provide OO have only weakened the once perfect encapsulation we enjoyed with C.

INHERITANCE?

If OO languages did not give us better encapsulation, then they certainly gave us inheritance.

Well—sort of. Inheritance is simply the redeclaration of a group of variables and functions within an enclosing scope. This is something C programmers³ were able to do manually long before there was an OO language.

Consider this addition to our original `point.h` C program:

[Click here to view code image](#)

`namedPoint.h`

```
struct NamedPoint;

struct NamedPoint* makeNamedPoint(double x, double y, char* name);
void setName(struct NamedPoint* np, char* name);
char* getName(struct NamedPoint* np);
```

[Click here to view code image](#)

`namedPoint.c`

```
#include "namedPoint.h"
#include <stdlib.h>

struct NamedPoint {
    double x,y;
    char* name;
};

struct NamedPoint* makeNamedPoint(double x, double y, char* name)
{
    struct NamedPoint* p = malloc(sizeof(struct NamedPoint));
    p->x = x;
    p->y = y;
    p->name = name;
    return p;
}

void setName(struct NamedPoint* np, char* name) {
    np->name = name;
}

char* getName(struct NamedPoint* np) {
    return np->name;
}
```

[Click here to view code image](#)

main.c

```
#include "point.h"
#include "namedPoint.h"
#include <stdio.h>

int main(int ac, char** av) {
    struct NamedPoint* origin = makeNamedPoint(0.0, 0.0, "origin");
    struct NamedPoint* upperRight = makeNamedPoint (1.0, 1.0,
"upperRight");
    printf("distance=%f\n",
        distance(
            (struct Point*) origin,
            (struct Point*) upperRight));
}
```

If you look carefully at the `main` program, you'll see that the `NamedPoint` data structure acts as though it is a derivative of the `Point` data structure. This is because the order of the first two fields in `NamedPoint` is the same as `Point`. In short, `NamedPoint` can masquerade as `Point` because `NamedPoint` is a pure superset of `Point` and maintains the ordering of the members that correspond to `Point`.

This kind of trickery was a common practice⁴ of programmers prior to the advent of OO. In fact, such trickery is how C++ implements single inheritance.

Thus we might say that we had a kind of inheritance long before OO languages were invented. That statement wouldn't quite be true, though. We had a trick, but it's not nearly as convenient as true inheritance. Moreover, multiple inheritance is a considerably more difficult to achieve by such trickery.

Note also that in `main.c`, I was forced to cast the `NamedPoint` arguments to `Point`. In a real OO language, such upcasting would be implicit.

It's fair to say that while OO languages did not give us something completely brand new, it did make the masquerading of data structures significantly more convenient.

To recap: We can award no point to OO for encapsulation, and perhaps a half-point for inheritance. So far, that's not such a great score.

But there's one more attribute to consider.

POLYMORPHISM?

Did we have polymorphic behavior before OO languages? Of course we did. Consider this simple C `copy` program.

[Click here to view code image](#)

```
#include <stdio.h>

void copy() {
    int c;
    while ((c=getchar()) != EOF)
        putchar(c);
}
```

The function `getchar()` reads from `STDIN`. But which device is `STDIN`? The `putchar()` function writes to `STDOUT`. But which device is that? These functions are *polymorphic*—their behavior depends on the type of `STDIN` and `STDOUT`.

It's as though `STDIN` and `STDOUT` are Java-style interfaces that have implementations for each device. Of course, there are no interfaces in the example C program—so how does the call to `getchar()` actually get delivered to the device driver that reads the character?

The answer to that question is pretty straightforward. The UNIX operating system requires that every IO device driver provide five standard functions:⁵ `open`, `close`, `read`, `write`, and `seek`. The signatures of those functions must be identical for every IO driver.

The `FILE` data structure contains five pointers to functions. In our example, it might look like this:

[Click here to view code image](#)

```
struct FILE {
    void (*open)(char* name, int mode);
    void (*close)();
    int (*read)();
    void (*write)(char);
    void (*seek)(long index, int mode);
};
```

The IO driver for the console will define those functions and load up a `FILE` data structure with their addresses—something like this:

[Click here to view code image](#)

```
#include "file.h"

void open(char* name, int mode) { /*...*/ }
void close() { /*...*/ };
int read() { int c; /*...*/ return c; }
void write(char c) { /*...*/ }
void seek(long index, int mode) { /*...*/ }

struct FILE console = {open, close, read, write, seek};
```

Now if `STDIN` is defined as a `FILE*`, and if it points to the console data structure, then `getchar()` might be implemented this way:

[Click here to view code image](#)

```
extern struct FILE* STDIN;

int getchar() {
    return STDIN->read();
}
```

In other words, `getchar()` simply calls the function pointed to by the `read` pointer of the `FILE` data structure pointed to by `STDIN`.

This simple trick is the basis for all polymorphism in OO. In C++, for example, every virtual function within a class has a pointer in a table called a `vtable`, and all calls to virtual functions go through that table. Constructors of derivatives simply load their versions of those functions into the `vtable` of the object being created.

The bottom line is that polymorphism is an application of pointers to functions. Programmers have been using pointers to functions to achieve polymorphic behavior since Von Neumann architectures were first implemented in the late 1940s. In other words, OO has provided nothing new.

Ah, but that's not quite correct. OO languages may not have given us polymorphism, but they have made it much safer and much more convenient.

The problem with explicitly using pointers to functions to create polymorphic